BEST EXECUTION | TALLER DE REINFORCEMENT LEARNING

Guía de coordinación del equipo

División del trabajo entre tres personas para avanzar en paralelo, reducir conflictos de integración y cubrir todos los ejercicios.

Objetivo operativo
Separar entorno, objetivo TWAP y entrenamiento adaptativo mediante interfaces claras. Cada persona será propietaria de módulos y notebooks distintos.

PERSONA 1	PERSONA 2	PERSONA 3
Entorno y ejecución completa **3,5 puntos**	TWAP y calibración **3,5 puntos**	Exploración y checkpoints **3,0 puntos**

Versión 1.0 | 7 de noviembre de 2026

1. Recomendación ejecutiva

No conviene repartir simplemente un conjunto de celdas del mismo notebook. Los ejercicios 2 y 3 dependen del entorno del ejercicio 1 y los ficheros *.ipynb* generan conflictos difíciles de resolver en Git. La propuesta combina un breve arranque conjunto con tres líneas de trabajo técnicamente independientes.

Antes de abrir tres ramas
Consolidar el estado actual del repositorio en un commit de referencia. En la revisión inicial se detectaron numerosos cambios locales, dos notebooks de solución eliminados y un fichero no versionado. Empezar sin esa fotografía común aumenta el riesgo de perder trabajo.

Principios de coordinación

- **Una persona por archivo.** Ningún módulo o notebook tiene dos propietarios durante la fase paralela.
- **Composición en lugar de duplicación.** El entorno recibe una estrategia de recompensa y el agente una política de exploración.
- **Interfaces congeladas pronto.** Tras el primer punto de integración no se cambian las firmas de `reset()` y `step()` sin acuerdo del equipo.
- **La motivación forma parte del entregable.** Cada persona documenta las decisiones de su propio bloque; no se delega toda la memoria a una cuarta fase.
- **Un solo activo.** Trabajar con REP reduce el coste experimental y satisface el alcance del ejercicio.

Resumen del reparto

Persona	Responsabilidad	Ejercicios	Entrega
1	Entorno y bucle completo	1 - 3,5 puntos	Entorno modular y notebook ejecutado
2	Objetivo TWAP y calibración	2 - 3,5 puntos	Reward, estado, experimentos y gráficos
3	Entrenamiento y exploración	3 + 4C - 3,0 puntos	Epsilon adaptativo, parada y checkpoints

Decisión de alcance
Prioridad: completar los ejercicios 1, 2 y 3, que suman 8,5 puntos. La idea 4C se implementa después, porque reutiliza directamente el trabajo de la persona 3.

Persona 1 - Entorno y ejecución completa

Alcance: Ejercicio 1 - 3,5 puntos

Construir una base estable que permita ejecutar episodios completos y que las otras dos personas puedan extender sin editar el notebook original.

Responsabilidades

- Extraer `BestExecutionEnv` del notebook a un módulo reutilizable.
- Implementar generación de episodios, estado, `reset()`, `step()`, acciones, VWAP y condiciones de parada.
- Parametrizar duración, `look_back`, tamaño CARE y semilla aleatoria.
- Definir la interfaz `RewardStrategy` sin implementar todavía el objetivo TWAP.
- Añadir pruebas rápidas para varios días, horas, duraciones y tamaños CARE.
- Entregar un notebook ejecutado que demuestre recolección de experiencias y un entrenamiento completo.

Archivos bajo su propiedad

src/environment/best_execution.py	Clase del entorno y ciclo del episodio.
src/environment/config.py	Configuración tipada de episodios.
tests/test_environment.py	Pruebas de estados, acciones y parada.
workshop/submission/exercise_1.ipynb	Demostración ejecutada del ejercicio 1.

Definición de terminado
Un episodio puede ejecutarse de principio a fin; las observaciones mantienen forma estable; el volumen CARE no se sobrepasa; y las condiciones de parada funcionan tanto por tiempo como por orden completada.

Contrato que debe publicar primero
La persona 1 debe subir en las primeras horas las clases y firmas, aunque inicialmente contengan implementaciones mínimas. Las personas 2 y 3 podrán desarrollar contra ese contrato o contra dobles de prueba.

Persona 2 - TWAP, recompensa y calibración

Alcance: Ejercicio 2 - 3,5 puntos

Conseguir que el agente aprenda una pauta temporal similar a TWAP, explicando cómo el estado y la recompensa producen ese comportamiento.

Responsabilidades

- Implementar un TWAP determinista como referencia y prueba de coherencia.
- Diseñar una estrategia de recompensa TWAP separada del entorno.
- Justificar las variables del estado: tiempo restante, volumen restante y, si aporta valor, desviación respecto al ritmo objetivo.
- Ajustar gamma, learning rate, buffer, batch y arquitectura, registrando la lógica de las decisiones.
- Probar distintos días, horas de inicio, duraciones y tamaños CARE sobre el conjunto de train.
- Generar gráficos de volumen acumulado, pauta temporal de acciones y comparación de VWAP.

Archivos bajo su propiedad

src/agents/twap.py	Política TWAP determinista de referencia.
src/rewards/base.py	Abstracción común de recompensa.
src/rewards/twap.py	Recompensa y métricas TWAP.
src/report/report.py	Gráficos y resúmenes experimentales.
workshop/submission/exercise_2.ipynb	Resultados y motivación del ejercicio 2.

Definición de terminado
El volumen se distribuye de forma aproximadamente equidistante; la orden CARE se completa; el comportamiento se mantiene al variar el episodio; y los hiperparámetros quedan motivados con evidencia observable.

Evitar trabajo innecesario
El enunciado no exige validación formal ni resultados perfectos. Sí exige funcionamiento con distintos días, momentos, duraciones y tamaños CARE. La robustez de parametrización importa más que una búsqueda exhaustiva de hiperparámetros.

Persona 3 - Entrenamiento y exploración adaptativa

Alcance: Ejercicio 3 + idea 4C - 3,0 puntos

Separar el entrenamiento del notebook y hacer que epsilon responda a la evolución del aprendizaje, no al contador de episodios.

Responsabilidades

- Extraer el bucle de entrenamiento a una clase `Trainer` reutilizable.
- Definir una interfaz `ExplorationPolicy` e inyectarla en `DDQNAgent`.
- Elegir una señal de mejora: recompensa media móvil, desviación TWAP, tasa de finalización o combinación estable.
- Implementar límites de epsilon, ventana de suavizado y tolerancia para evitar reacciones al ruido.
- Añadir parada temprana, guardado periódico de pesos y recuperación del mejor entrenamiento.
- Unificar la configuración de Python y dependencias para asegurar ejecuciones reproducibles.

Archivos bajo su propiedad

src/training/trainer.py	Orquestación, métricas y callbacks.
src/agents/exploration.py	Políticas de exploración adaptativa.
src/agents/dqn.py	Integración de la política en el agente.
tests/test_exploration.py	Límites y reacción a mejora/estancamiento.
workshop/submission/exercise_3_4.ipynb	Evidencia de ejercicios 3 y 4C.

Definición de terminado
Epsilon cambia únicamente cuando cambia la señal de aprendizaje; se mantiene dentro de sus límites; la lógica está explicada; y un checkpoint guardado puede cargarse para continuar o evaluar el modelo.

2. Arquitectura compartida

La separación siguiente permite que recompensa, exploración y entorno evolucionen sin copiar código ni introducir subclases casi idénticas.

BestExecutionEnv	Gestiona episodios, estado, acciones y estadísticas. Delega el cálculo del reward.
RewardStrategy	Convierte el estado de ejecución en una recompensa. TWAP es una implementación concreta.
DDQNAgent	Selecciona acciones y aprende valores Q. Delega la decisión exploratoria.
ExplorationPolicy	Actualiza epsilon a partir de métricas de aprendizaje.
Trainer	Coordina entorno, agente, replay buffer, red objetivo, métricas y checkpoints.

Contrato mínimo acordado

```
env = BestExecutionEnv(data, config, reward_strategy)
state = env.reset(seed=seed)
next_state, reward, done, info = env.step(action)

agent = DDQNAgent(env, exploration_policy=policy)
result = trainer.train(agent, env, callbacks=[checkpoint])
```

Regla de integración
Tras congelar este contrato, una modificación incompatible requiere acuerdo de las tres personas y actualización simultánea de pruebas y notebooks.

3. Plan temporal

1. **Arranque conjunto - 60 minutos.** Crear commit de referencia, fijar entorno, elegir REP, acordar interfaces, semillas y métricas.
2. **Primer bloque paralelo.** Persona 1 implementa el entorno; persona 2 desarrolla TWAP y reward contra el contrato; persona 3 desarrolla trainer y exploración con un entorno mínimo.
3. **Punto de integración.** Ejecutar un episodio aleatorio y otro TWAP; congelar `reset()`, `step()` y las claves de `info`.
4. **Segundo bloque paralelo.** Cada persona completa pruebas, experimentos, gráficos y motivación en su notebook exclusivo.
5. **Cierre conjunto.** Ejecutar los notebooks desde cero, revisar criterios de aceptación y generar la entrega final.

4. Git e integración

Ramas sugeridas

feature/environment	Persona 1 - entorno, configuración y ejercicio 1.
feature/twap-reward	Persona 2 - baseline TWAP, reward, reporting y ejercicio 2.
feature/adaptive-exploration	Persona 3 - trainer, exploración, checkpoints y ejercicios 3-4.

Orden recomendado de integración

1. Entorno, configuración y contrato de recompensa.
2. Trainer, política de exploración e integración con DDQN.
3. Reward TWAP, reporting y experimentos.
4. Notebooks ejecutados, memoria y limpieza final.

> **Ficheros compartidos sensibles**
> `setup.py`, requirements, README y configuración del contenedor tendrán un único propietario temporal: la persona 3. El resto del equipo propone cambios mediante revisión, pero no los edita en paralelo.

5. Criterios de aceptación

Ejercicio 1

- Episodio completo sin errores ni observaciones con forma variable.
- Replay buffer, aprendizaje y actualización de la red objetivo en el orden correcto.
- Finalización por tiempo u orden completada y penalización coherente del volumen pendiente.

Ejercicio 2

- Ejecución aproximadamente equidistante y orden CARE completada.
- Comportamiento consistente al variar día, inicio, duración y volumen.
- Gráficos legibles y relación explícita entre estado, reward y conducta aprendida.

Ejercicio 3 y 4C

- Epsilon depende de una señal de mejora o conocimiento, no del número de episodio.
- Política acotada, estable frente al ruido y explicada con métricas.
- Checkpoint recuperable y condición razonada de parada del entrenamiento.

Entrega completa

- Tres notebooks ejecutados de principio a fin y sin marcadores TODO pendientes.
- Cada bloque modificado está identificado y cada decisión relevante está motivada.
- No hay lógica duplicada entre notebooks y módulos Python.
- Un compañero distinto del autor ha revisado cada bloque.

6. Agenda de arranque

Esta agenda permite que el equipo empiece a producir código independiente durante la primera sesión.

00:00-00:10	Revisar el estado de Git y crear el commit de referencia.
00:10-00:20	Elegir Python, dependencias y método común de ejecución.
00:20-00:35	Acordar interfaces, configuración de episodio y claves de métricas.
00:35-00:45	Crear ramas, estructura de paquetes y notebooks separados.
00:45-01:00	Publicar stubs, ejecutar un smoke test y comenzar el trabajo paralelo.

Cadencia mínima

- Un mensaje breve al inicio del día: objetivo, archivos previstos y bloqueo principal.
- Un punto de integración diario de 15 minutos con una demostración ejecutable, no solo una explicación.
- Pull requests pequeños y revisables; evitar acumular todos los cambios hasta el final.
- Cada persona revisa el bloque de otra: 1 revisa a 2, 2 revisa a 3 y 3 revisa a 1.

> **Resultado esperado**
> Tres líneas de trabajo equilibradas por puntuación, notebooks sin conflictos y una arquitectura reutilizable que permite integrar entorno, TWAP y exploración adaptativa con cambios localizados.

Documento preparado a partir del enunciado de ejercicios del taller de RL y de la estructura actual del repositorio best-execution.